

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Campus Coração Eucarístico
Curso de Engenharia de Computação

Disciplina: Tópicos III – Empreendedorismo e Inovação

Documentação Técnica de Software:

PetCuida Android

Arquitetura, Configuração e Requisitos de Evolução

Integrantes:

Gabriel da Silva Cassino
Giovanna Naves Ribeiro
Júlia Rodrigues Vasconcellos Melo
Kathleen Rodrigues Leite Barbosa

Professor-orientador:
João Carlos Oliveira Caetano
Professor-tutor:
Paulo Mattos

Belo Horizonte
2026

Gabriel da Silva Cassino
Giovanna Naves Ribeiro
Júlia Rodrigues Vasconcellos Melo
Kathleen Rodrigues Leite Barbosa

Documentação Técnica de Software: PetCuida Android

(Especificação da Aplicação Mobile — Revisão `D4C2B_LGPD`)

Documentação técnica apresentada à disciplina Tópicos III – Empreendedorismo e Inovação, do Curso de Engenharia de Computação da PUC Minas, como artefato complementar de engenharia referente à arquitetura mobile do projeto PetCuida.

Belo Horizonte
2026

Sumário

1 Identificação e arquitetura **2**

2 Configuração Android observada **2**

3 Fluxos e tratamento de dados **2**

4 Execução e validação **3**

5 Segurança e requisitos de evolução **3**

6 Arquivos relevantes **4**

Referências **4**

1 Identificação e arquitetura

O **PetCuida Android** constitui o destino nativo para dispositivos Android desenvolvido a partir da mesma base de código Flutter utilizada na versão Web. O código-fonte em Dart reside no diretório `lib/`, enquanto as configurações específicas da plataforma nativa encontram-se em `android/`. O identificador de pacote declarado no Gradle é `com.grupo4topicos3.petcuida.petcuida`, utilizando o rótulo de exibição `PetCuida` e a versão `1.3.0+1` definida no arquivo `pubspec.yaml`.

A camada nativa inicializa a classe `MainActivity` utilizando o padrão *Flutter embedding v2*. As rotas, o gerenciamento de estado e os componentes visuais (*widgets*) são integralmente compartilhados com a aplicação Web. Na entrega atual, a camada de dados opera por meio de uma simulação em memória volátil (*mock*), e a interface informa explicitamente ao usuário que a autenticação e as integrações externas são demonstrativas.

2 Configuração Android observada

O arquivo `android/app/build.gradle.kts` herda dinamicamente as propriedades `compileSdk`, `targetSdk`, `minSdk`, versionamento e NDK diretamente da configuração do Flutter, fazendo com que os valores efetivos dependam do SDK instalado no ambiente de compilação. A compatibilidade de compilação para Java e Kotlin está configurada para a **JVM 17**.

O manifesto Android (`AndroidManifest.xml`) define o rótulo `PetCuida`, configura a *activity* principal como exportada para o inicializador do sistema (*launcher*) e declara a permissão de acesso à câmera. O recurso de hardware de câmera foi marcado como opcional (`required="false"`) para assegurar que o aplicativo possa ser instalado em dispositivos que não possuam câmera integrada.

Ressalta-se que o bloco de compilação `release` no Gradle está atualmente configurado para assinar o pacote com a chave padrão de depuração (*debug*). Essa configuração atende apenas a testes internos e não constitui um perfil distribuível em lojas oficiais. Adicionalmente, o pacote de entrega omite o arquivo `local.properties` (que armazena caminhos locais da máquina de desenvolvimento) e não inclui credenciais produtivas do Firebase.

3 Fluxos e tratamento de dados

A experiência do protótipo contempla as telas de abertura (*splash*), autenticação e cadastro simulados, *onboarding*, painel principal, perfil do tutor, gestão de pets, triagem orientativa, rede solidária, agendamentos e painel corporativo para organizações parceiras. Tanto o cadastro quanto a sessão ativa não são persistidos em banco de dados nem vinculados a um provedor de identidade real. Todas as ações executadas alteram apenas o estado em memória, sendo reinicializadas ao encerrar o processo do aplicativo.

A permissão de câmera atende exclusivamente ao fluxo de seleção de imagem para o perfil do animal, sem realizar envio (*upload*) para servidores remotos. Não há sincronização em nuvem de prontuários, processamento de pagamentos, saldo financeiro real, envio de notificações *push* ou integração ativa com clínicas. Por fim, o módulo de triagem atua estritamente como guia orientativo de interface, não possuindo caráter de diagnóstico veterinário.

4 Execução e validação

Para compilar, testar e executar o projeto em um ambiente configurado com Flutter SDK, Android SDK, Java 17 e um dispositivo físico ou emulador, devem-se executar os seguintes comandos na raiz do projeto Flutter:

- `flutter pub get` — instala e resolve as dependências declaradas;
- `flutter analyze` — executa a análise estática de código e verificação de tipos;
- `flutter test` — executa a suíte de testes unitários e de *widgets*;
- `flutter run -d <id-do-dispositivo>` — inicia a aplicação em modo de depuração;
- `flutter build apk --debug` — gera o pacote instalável (APK) para testes internos.

Antes de gerar uma versão de produção (*release*), é indispensável configurar uma chave de assinatura segura e verificar o versionamento, as versões efetivas do SDK, as permissões declaradas e a árvore de dependências. Nota-se que o *Gradle wrapper* isoladamente não compila o projeto sem a presença do SDK do Flutter no ambiente.

5 Segurança e requisitos de evolução

O APK de demonstração não deve ser utilizado com dados pessoais reais nem implantado como serviço ativo. Para a transição rumo ao piloto controlado, estabelecem-se os seguintes requisitos técnicos obrigatórios:

1. **Autenticação e controle de acesso:** Implementar autenticação real com sessão persistente, derivar a propriedade dos registros a partir de identidade confiável, testar o isolamento entre contas e aplicar políticas de segurança do tipo *default-deny* no banco de dados.
2. **Saneamento de ambiente:** Remover dados iniciais de demonstração (*seeds*) dos ambientes remotos e migrar o modelo de dados para a camada produtiva.
3. **Conformidade com a LGPD:** Implementar mecanismos formais de gestão de consentimento, controle de armazenamento de imagens, políticas de retenção e exclusão de dados, além de exibir mensagens claras de falha de rede sem simular falso sucesso.
4. **Distribuição e hardware:** Configurar assinatura criptográfica de *release* protegida fora do repositório de código e reavaliar a real necessidade da permissão de câmera, mantendo-a opcional.
5. **Qualidade e resiliência:** Validar o comportamento de formulários, teclado virtual, escalonamento de fontes, rotação de tela, ciclo de vida da aplicação (retorno após suspensão) e recuperação de conectividade em dispositivos reais, além de executar testes de autorização no backend antes de inserir credenciais de produção.

6 Arquivos relevantes

A Tabela 1 relaciona os principais arquivos estruturais que compõem a implementação Android e a camada compartilhada do aplicativo.

Tabela 1: Principais arquivos da implementação Android e da camada compartilhada.

Arquivo	**Função**
`android/app/src/main/AndroidManifest.xml`	Define o rótulo, o ponto de entrada (*launcher*) e as permissões do sistema.
`android/app/build.gradle.kts`	Especifica a identidade do pacote e as opções de compilação Android.
`android/app/src/main/kotlin/../MainActivity.kt`	Implementa a *activity* nativa hospedeira do Flutter.
`lib/main.dart`	Ponto de entrada e raiz de composição compartilhada da aplicação.
`lib/core/services/app_data_service.dart`	Gerencia o estado e os dados simulados da demonstração.
`lib/core/services/auth_service.dart`	Implementa o fluxo de autenticação demonstrativa sem backend.
`test/core/services/app_data_service_test.dart`	Contém os casos de testes unitários da camada de serviços.

Referências

[1] GRUPO 4. **Documentação Própria A: código-fonte e configuração Android (`flutter/`)**. Belo Horizonte: PUC Minas, 2026.

[2] GRUPO 4. **Pacote de entrega `PetCuida_D4C2B_LGPD` e Protótipos Iniciais**. Belo Horizonte: PUC Minas, 2026.

[3] GRUPO 4. **Documentação Própria B: proposta técnica e critérios de aceite para Web e Mobile**. Belo Horizonte: PUC Minas, 2026.